تقرير بحثي مصغر

الذكاء الاصطناعي التوليدي كعامل مساعد للمطورين

المقرر: تطبيقات الإنترنت والمنصات الإلكترونية

[رنا هشام حسين حسين] **:اسم الطالب**

[2502681] **:الرقم الجامعي**

[مها زغلول] **:اسم الدكتور**

أولاً: المقدمة

الذكاء الاصطناعي التوليدي (Generative AI) فرع من الذكاء الاصطناعي تُدرَّب فيه النماذج على كميات ضخمة من البيانات، ثم تُستخدم لإنتاج محتوى جديد، مثل النصوص والصور والشيفرات البرمجية، بدلاً من الاكتفاء بتصنيف البيانات أو التنبؤ بها. وتقوم النماذج اللغوية الكبيرة (Large Language Models) مثل ChatGPT وGemini على بنية المحوّلات (Transformers)، وهي بنية تتيح للنموذج توقّع الكلمة أو الرمز التالي بناءً على السياق السابق، ولذلك يستطيع التعامل مع اللغة الطبيعية ولغات البرمجة على حد سواء.

في مجال تطوير الويب، أصبحت هذه النماذج أداة شائعة بين المطورين. فهي تقرأ الشيفرة وتشرحها، وتقترح تعديلات عليها، وتساعد في تتبع الأخطاء. وتزداد فائدتها عند المبتدئين، لأنها تتيح لهم طرح سؤال بلغتهم الخاصة والحصول على شرح فوري يناسب مستواهم، دون انتظار مدرّس أو البحث بين صفحات توثيق طويلة. ويهدف هذا التقرير إلى بيان كيف يمكن توظيف هذه النماذج في فهم أكواد الويب الأساسية وشرحها، مع التنبيه على حدود هذا الاستخدام.

ثانياً: عرض الموضوع

١. دور ChatGPT وGemini في تفسير لغات الويب الأساسية

تقوم صفحات الويب على ثلاث لغات أساسية: HTML التي تصف بنية الصفحة ومحتواها، وCSS التي تتحكم في مظهرها وتنسيقها، وJavaScript التي تضيف السلوك والتفاعل. ويجد المبتدئ صعوبة في الربط بين هذه الطبقات الثلاث، وفي فهم سبب عمل الشيفرة بالشكل الذي تعمل به. وهنا يمكن لنماذج مثل ChatGPT وGemini أن تؤدي عدة أدوار:

- **تفسير الشيفرة سطراً بسطر:** يلصق المتعلم المقطع البرمجي ويطلب شرحاً مبسطاً لكل سطر ولوظيفته.
- **تعريف المصطلحات:** مثل الوسم (Tag) والسمة (Attribute) والمحدِّد (Selector) والحدث (Event)، مع أمثلة قصيرة لكل منها.
- **الربط بين اللغات الثلاث:** كأن يوضح النموذج كيف يختار JavaScript عنصراً من HTML ثم يغيّر مظهره المعرَّف في CSS.
- **التكيّف مع مستوى السائل:** يمكن طلب الشرح بأسلوب مبسط لمبتدئ تماماً، أو بتفصيل أعمق لمن لديه خلفية سابقة.

وقد أشارت دراسات في تعليم البرمجة إلى أن الشروح التي تولّدها النماذج اللغوية للشيفرة كانت في الغالب مفهومة ومفيدة للطلاب، وإن لم تخلُ من أخطاء أحياناً (MacNeil et al., 2023). كما أن الجودة تتفاوت بحسب صياغة السؤال، فكلما كان الطلب أوضح وأكثر تحديداً جاءت الإجابة أدق.

٢. مثال عملي: شرح كود بسيط خطوة بخطوة

لنفترض أن مبتدئاً صادف المقطع التالي، وهو زر يعرض رسالة عند الضغط عليه:

```
<button id="btn">Click me</button>
<p id="msg"></p>

<script>
  const btn = document.getElementById("btn");

  btn.addEventListener("click", function () {
    document.getElementById("msg").textContent = "Hello!";
  });
</script>
```

يمكن للمتعلم أن يكتب للنموذج طلباً مثل: «اشرح لي هذا الكود سطراً بسطر كأنني مبتدئ لا أعرف لغة JavaScript». وعندها يُتوقع أن يفكك النموذج الكود على النحو الآتي:

١. **السطر الأول:** الوسم `button` ينشئ زراً في الصفحة، والسمة `id` تمنحه اسماً فريداً هو `btn`، ما يسمح لـ JavaScript بالوصول إليه لاحقاً.

٢. **السطر الثاني:** الوسم `p` ينشئ فقرة فارغة معرّفها `msg`، وهي المكان الذي ستظهر فيه الرسالة.

٣. **الوسم** `script` : يحدد بداية شيفرة JavaScript داخل الصفحة.

٤. **الدالة** `getElementById` : تبحث في مستند الصفحة عن العنصر الذي معرّفه `btn` وتعيده.

٥. **الدالة** `addEventListener` : تُخبر المتصفح بأن ينتظر الحدث `click` (النقر على الزر)، وأن ينفذ الدالة المرفقة عند وقوعه.

٦. **الخاصية** `textContent` : تضع النص فتظهر `Hello!`، داخل الفقرة التي معرّفها `msg`، الرسالة للمستخدم.

وبعد هذا الشرح يستطيع المتعلم متابعة الحوار بأسئلة إضافية، مثل السؤال عن الفرق بين `textContent` و`innerHTML`، أو عمّا يحدث إذا استُبدل الحدث `click` بالحدث `dblclick`. وهذا التفاعل المتدرج هو ما يميز الاستعانة بالنموذج عن قراءة مرجع ثابت، إذ يتحول التعلم إلى حوار يتكيف مع تساؤلات المتعلم.

٣. فوائد الاستعانة بالذكاء الاصطناعي

تسريع التعلم. يوفّر النموذج شرحاً فورياً دون الحاجة إلى البحث في عدة مصادر، ما يقلل الوقت الضائع عند التعثر في نقطة بسيطة. وقد وجدت دراسة أُجريت على متعلمين مبتدئين أن استخدام مولّدات الشيفرة ارتبط بتحسن في إنجاز المهام البرمجية دون أثر سلبي ظاهر في الأداء اللاحق، مع ملاحظة أن النتائج تتفاوت بين المتعلمين (Kazemitabaar et al., 2023).

تصحيح الأخطاء (Debugging). يستطيع المبتدئ لصق الشيفرة مع رسالة الخطأ وسؤال النموذج عن سببها. فمثلاً قد يكتب وسم إغلاق ناقصاً في HTML، أو يخطئ في اسم المتغير في JavaScript، فيشير النموذج إلى موضع الخطأ ويشرح سببه وكيفية إصلاحه، وهو ما يساعد على تعلّم نمط الأخطاء الشائعة وليس إصلاحها فقط.

شرح الأكواد المعقدة. عند مواجهة شيفرة طويلة كتبها شخص آخر، يمكن طلب تلخيص لوظيفتها العامة أولاً، ثم تفصيل الأجزاء الغامضة منها، وهذا يسهّل على المبتدئ قراءة مشاريع حقيقية والتعلم منها.

توفير أمثلة إضافية. يمكن طلب أمثلة متعددة للمفهوم نفسه بمستويات مختلفة، وهذا يعزز الفهم عن طريق التكرار والتنويع.

٤. المحاذير واشتراطات الاستخدام

رغم هذه الفوائد، فإن الاستخدام الواعي يتطلب الانتباه إلى الأمور الآتية:

- **عدم الاعتماد الكلي:** النماذج التوليدية قد تنتج إجابات تبدو مقنعة لكنها خاطئة أو غير مكتملة، ولا تضمن دقة المعلومة. وقد بيّنت دراسات أن الشيفرة التي تولّدها هذه الأدوات قد تتضمن ثغرات أمنية في بعض الحالات (Pearce et al., 2022).
- **التحقق من صحة الكود:** ينبغي تجربة الشيفرة فعلياً في المتصفح، ومقارنة الشرح بالتوثيق الرسمي مثل MDN Web Docs، خصوصاً في الخصائص الحديثة أو التي تتغير مع الوقت.
- **فهم الأساسيات أولاً:** إذا اعتمد المبتدئ على النموذج قبل أن يتعلم أساسيات HTML وCSS وJavaScript، فقد ينسخ الحلول دون فهمها، فيضعف تكوين مهارة حل المشكلات لديه. والأفضل أن يحاول الحل بنفسه أولاً، ثم يستعين بالنموذج للمراجعة أو التوضيح.
- **صياغة الأسئلة بدقة:** كلما حدد المتعلم لغته ومستواه والنتيجة المطلوبة، تحسنت جودة الإجابة.
- **مراعاة النزاهة الأكاديمية والخصوصية:** يجب الالتزام بسياسات المؤسسة التعليمية بشأن استخدام هذه الأدوات في الواجبات، وتجنب إدخال بيانات حساسة أو مفاتيح وصول خاصة في المحادثات.

ثالثاً: الخاتمة

تمثل نماذج الذكاء الاصطناعي التوليدي مثل ChatGPT وGemini وسيلة داعمة لمتعلمي تطوير الويب، لما توفره من شرح تفاعلي للشيفرة وتصحيح للأخطاء وتبسيط للمفاهيم. غير أن قيمتها تعتمد على طريقة استخدامها، فهي أداة مساعدة لا بديلاً عن التعلم المنهجي ولا عن المصادر الموثوقة. والاستخدام الأمثل يجمع بين الاستفادة من سرعة النموذج، والتحقق المستمر من مخرجاته، وبناء فهم شخصي راسخ للأساسيات.

رابعاً: المراجع

Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. arXiv:2107.03374.

DEV Community. (n.d.). DEV Community: Web development articles and tutorials. https://dev.to

Flanagan, D. (2020). JavaScript: The definitive guide (7th ed.). O'Reilly Media.

Gemini Team, Google. (2023). Gemini: A family of highly capable multimodal models. arXiv:2312.11805.

Haverbeke, M. (2018). Eloquent JavaScript (3rd ed.). No Starch Press.

Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems. ACM.

MacNeil, S., Tran, A., Mogil, D., Bernstein, S., Ross, E., & Huang, Z. (2023). Experiences from using code explanations generated by large language models in a web software development e-book. Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023). ACM.

Mozilla Developer Network. (n.d.). MDN Web Docs. https://developer.mozilla.org

OpenAI. (2023). GPT-4 technical report. arXiv:2303.08774.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. 2022 IEEE Symposium on Security and Privacy (SP). IEEE.

Vaswani, A., Shazeer, N., Parmar, N., et al. (2017). Attention is all you need. Advances in Neural Information Processing Systems, 30.